Підготовка до заняття 8

RAG системи production level

r_d

Про що цей текст

На занятті 7 ми зробили retrieval: hybrid search, який за питанням повертає релевантні чанки, і довели його якість через recall@k. Сьогодні друга половина системи: з цих чанків зібрати промпт, отримати від моделі відповідь з посиланнями на джерела, перевірити її кодом і не показувати користувачу вигадку, коли в документах відповіді немає.

Текст розрахований на розробника, який уже викликав LLM через API, знає structured output і версіонування промптів (заняття 3–4) і бачив чанки з метаданими (заняття 6). Усі цифри взято з прогону практики: 69 чанків вигаданої компанії Northwind Logistics, 23 питання з еталонами, модель `gpt-4o-mini` через OpenRouter. Кожен розділ закінчується блоком «Головне», посилання «Глибше» необов'язкові. Читати 60–75 хвилин.

План

1. Навіщо RAG
модель не бачила ваших документів

2. RAG чи fine-tuning
дані в конфігу чи в бінарнику

3. Архітектура
етапи, контракти, оптимізація retrieval

4. Контекст і цитування
формат джерел, схема відповіді, валідація

5. «Не знаю»
чому поріг не працює, три рівні захисту

6. Версії й конфлікти
старий регламент проти нової пам'ятки

7. Контекст як ресурс
k, токени, вартість, latency

8. Більше, ніж вікно
map-reduce, підсумки, long context

9. Діалог і пам'ять
condense question, історія як ресурс

10. Де зламалось
трасування помилки по етапах

11. Перевірте себе
питання для самоперевірки

12. Перед заняттям
середовище і домашня частина

Що ви вмітимете після заняття

- будувати RAG-пайплайн, який відповідає з цитатами і повертає «не знайшов», коли відповіді немає;
- локалізувати помилку: retrieval, бюджет контексту, дані чи генерація;
- обирати розмір контексту за вимірами якості, вартості й latency;
- відповідати на запити, яким потрібен цілий документ, а не top-k.

Терміни, які знадобляться

Термін	Що означає в цьому тексті
retrieval, top-k	пошук з заняття 7 (BM25 + вектори + RRF), який повертає k найрелевантніших чанків
контекст, бюджет	джерела, які ми вставляємо в промпт, і ліміт токенів на них
grounding, цитата	відповідь спирається на конкретні джерела з контексту і посилається на них мітками S1, S2…
галюцинація	впевнена відповідь, якої немає в джерелах: вигадане правило, число чи посилання
abstain / false abstain	частка правильних «не знайшов» на питаннях без відповіді / частка помилкових «не знайшов» на питаннях, де відповідь є
retrieved / in_context	частка питань, де правильний чанк є серед k знайдених / дійшов до промпту після бюджету
correct / cited	частка відповідей з усіма еталонними фактами / частка відповідей, що цитують правильний чанк

РОЗДІЛ 1

1 Навіщо RAG

1.1 Модель не бачила ваших документів

Ми спитали `gpt-4o-mini` без документів: «I work at Northwind Logistics. How much can I spend on a hotel per night during a business trip?» Модель не вигадала, але й не допомогла: «The amount… typically depends on your company's travel policy… check with your HR department». Відповідь (120 EUR за ніч, погоджує тімлід) лежить у таблиці лімітів `expense_policy.md`, а цього файлу не було в навчальних даних моделі.

Наївне рішення — покласти в промпт усі документи. На 69 чанках це ще можливо. На корпусі з тисяч документів запит не влізе у вікно, кожен виклик коштуватиме як тисячі токенів, а якість, як побачимо в розділі 7, падає з довжиною контексту.

1.2 Retrieval-augmented generation

RAG складається з двох етапів, які ви вже вмієте робити окремо: retrieval знаходить кілька релевантних чанків, а модель відповідає лише з них. Архітектурно це той самий патерн, що й звичайний бекенд-ендпоінт: спершу дістати дані зі сховища, потім відрендерити відповідь. Різниця в тому, що «рендерер» тут недетермінований, тому його вихід треба валідувати (розділи 4–5).

З тим самим питанням наш пайплайн відповідає «You can spend up to 120 EUR per night on a hotel during a business trip» з посиланням на `handbook/expense_policy.md#2`. Для внутрішнього асистента посилання важливіше за саму відповідь: користувач може відкрити джерело й перевірити.

1.3 Де RAG працює в продакшні

Типові сценарії — внутрішні асистенти і підтримка: відповіді вже записані у вікі, тікетах чи політиках, але їх не знаходять. Без вимірювань не видно, яка частка відповідей справді допомагає, тому якість ми міряємо з першого прогону.

ГОЛОВНЕ

Модель не знає ваших документів, тому RAG спершу **знаходить** релевантні чанки, а потім просить модель відповісти **лише з них** з посиланням на джерело. Retrieval з заняття 7 — перша половина, сьогодні робимо другу.

Глибше: Lewis et al.: Retrieval-Augmented Generation (2020)

РОЗДІЛ 2

2 RAG чи fine-tuning

2.1 Знання проти поведінки

Fine-tuning донавчає ваги моделі на ваших прикладах. Він добре змінює поведінку: формат відповіді, тон, термінологію, класифікацію. Нові факти він засвоює погано. У дослідженні Microsoft (Ovadia et al., 2023) Mistral 7B питали про події після дати навчання. Без допомоги модель відповідала правильно у 48.1% випадків. Після fine-tuning на текстах про ці події — у 50.4% (з перефразованими копіями текстів 58.8%). RAG на тих самих текстах дав 87.5%.

Аналогія з розробки

Fine-tuning схожий на те, щоб захардкодити довідкові дані в бінарник: працює, поки дані не змінились, а кожна зміна вимагає нового білда і не дає гарантії, що старе значення ніде не лишилось. RAG — це читання тих самих даних з бази під час запиту: оновили запис, і наступний запит уже бачить нове значення.

2.2 Економіка змін

Політика віддаленої роботи Northwind змінилась 1 вересня 2026 року (`Effective 2026-09-01. Replaces the version of 2026-01-15`). У RAG це переіндексація одного документа: кілька чанків, для незмінених embeddings беремо з кешу, займає секунди. Для донавченої моделі — новий датасет, нове тренування і перевірка, що стара версія справді «забулась».

Вимога	RAG	Fine-tuning
Оновити чи додати факт	переіндексувати документ	датасет і тренування
Показати джерело	чанк відомий	факт розчинений у вагах
Права доступу	фільтр у запиті до індексу (заняття 7)	модель «знає» все, на чому вчилась
Видалити дані (GDPR)	видалити чанки з індексу	перетренувати модель
Вартість запиту	довший промпт: ~850 токенів у практиці	короткий промпт, але дороге тренування
Формат, стиль, жаргон	інструкції й приклади в промпті	сильна сторона

У практиці одна відповідь з промптом v2 коштує в середньому $0.00015, тобто близько $0.15 за 1000 питань (ціни `gpt-4o-mini`: $0.15 за мільйон вхідних і $0.60 за мільйон вихідних токенів; перевіряйте актуальні). Підходи поєднуються: fine-tuning на формат відповідей, факти через RAG. Повний фреймворк вибору — на занятті 16.

ГОЛОВНЕ

Fine-tuning змінює **поведінку**, RAG дає **знання**. Факти, які змінюються, потребують посилань на джерело чи прав доступу, тримають в індексі, а не у вагах: оновлення — це переіндексація, а не тренування.

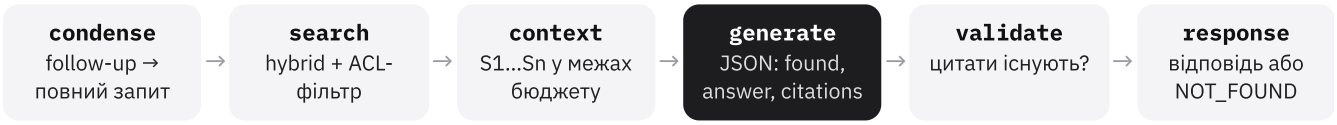
Глибше: Ovadia et al.: Fine-Tuning or Retrieval? (2023)

РОЗДІЛ 3

3 Архітектура

3.1 Offline і online

Індексація — офлайн-джоба: документи → чанки → embeddings → індекс (заняття 6–7). Вона запускається при зміні документів. Відповідь — онлайн-шлях, що виконується на кожен запит, і саме його будуємо сьогодні. Індекс беремо готовий: `retrieval.py` у практиці містить розв'язок заняття 7 з методом `search(query, groups, k)`.



Модель викликається лише у generate і, для уточнювальних питань, у condense. Решта — детермінований код з явними контрактами, який покривається юніт-тестами без ключа і без витрат: так влаштований `check.py`, що підставляє фейкову модель. Принцип той самий, що з будь-яким зовнішнім недетермінованим сервісом: модель пропонує, код валідує і вирішує, що піде користувачу.

```
build_context(hits, budget) -> (context: str, sources: dict[label, chunk])
generate(llm, question, context, system) -> Answer(found, answer, citations)
validate_citations(answer, sources) -> (Answer, invalid_labels)
rag(llm, retriever, question, groups, history=None) -> dict   # склеює все разом
```

3.2 Шлях одного запиту

Етап	«My work laptop was stolen on a trip. What do I do?»
search	5 чанків, першим `remote_work_policy_v2.docx#5` (Information security)
context	5 джерел; у середньому по набору 488 токенів контексту на запит
generate	`{"found": true, "answer": "You must report the stolen laptop to the IT service desk within one hour…", "citations": ["S1"]}`
validate	S1 є серед джерел → відповідь і посилання на документ

По всьому набору з 23 питань запит з промптом v2 займає в середньому ~850 вхідних токенів і 1.2–1.4 с. Майже весь час іде на виклик моделі, retrieval займає мілісекунди (заняття 7).

3.3 Оптимізація retrieval для RAG

Якість RAG обмежена зверху retrieval: якщо правильного чанка немає в контексті, модель або поверне NOT_FOUND, або вигадає. Тому поруч з метриками відповіді ми міряємо retrieval усередині пайплайна: `retrieved` — правильний чанк є серед k знайдених, `in_context` — він дійшов до промпту після бюджету. У практиці з k=5 обидві метрики 0.95: пропущено лише одне питання (q09, розділ 10.2). Різниця між `retrieved` і `correct` показує, скільки помилок припадає на генерацію і дані, а не на пошук.

Прийом	Що робить	Що відомо про ефект і ціну
Hybrid + RRF	BM25 і векторний пошук, об'єднані за рангами; є в практиці	заняття 7: recall@5 1.00 проти 0.94 у vector-only
Reranker	cross-encoder пересортовує top-20 кандидатів	заняття 7: MRR 0.82 → 0.88, але ~450 мс на запит і загублене українське питання
Contextual chunk headers	до чанка дописують, з якого він документа і про що (у практиці — шлях розділу з метаданих)	Anthropic: −35% провалів пошуку, з contextual BM25 −49%, з reranker −67%
Переписування запиту	follow-up → самостійний запит; є в практиці (TODO 9)	розділ 9
Multi-query	модель генерує кілька переформулювань, результати зливають через RRF	вищий recall на неоднозначних запитах; ціна — виклик моделі і кілька пошуків на запит
HyDE	модель пише гіпотетичну відповідь, шукаємо за її embedding	допомагає, коли запит короткий, а документи довгі; ризик — вигадка в гіпотетичній відповіді тягне не ті чанки
Small-to-big (parent-child)	шукаємо по дрібних чанках, у контекст віддаємо батьківський розділ	точний пошук і повний контекст; більше токенів на запит
Фільтри метаданих	ACL, версії, тип документа на рівні запиту до індексу	заняття 7, розділ 6

Кожен прийом додає latency, виклики моделі або токени. Тому додають їх тоді, коли `retrieved` і `in_context` показують, що втрачається саме пошук, і перевіряють прогоном, як на занятті 7. У нашій практиці retrieval уже дає 0.95, а єдиний промах при k=10 зникає з пошуку, але лишається проблемою даних (розділ 10.2). Оптимізувати пошук далі тут не має сенсу.

ГОЛОВНЕ

RAG — це пайплайн **search → context → generate → validate**. Модель — один-два виклики, решта — детермінований код, який тестується без моделі і має останнє слово. Якість обмежена retrieval, тому **retrieved і in_context** міряють поруч з якістю відповіді, а прийоми оптимізації пошуку додають лише під виміряну проблему.

Глибше: Anthropic: Contextual Retrieval · Gao et al.: HyDE

РОЗДІЛ 4

4 Контекст і цитування

4.1 Формат джерела

Модель може процитувати лише те, що має ідентифікатор. Тому кожен чанк у контексті отримує коротку мітку і заголовок з документом, розділом і датою:

```
[S1] policies/remote_work_policy_v2.docx · Remote Work Policy > 5. Information security · effective 2026-09-01
Company laptops must use full disk encryption and the corporate VPN ... Lost or stolen devices are reported to the IT service desk within one hour.

[S2] policies/remote_work_policy_v2.docx · Remote Work Policy > 4. Equipment · effective 2026-09-01
...
```

Чому `S1`, а не `chunk_id` на кшталт `operations/legacy_warehouse_manual_rev7.pdf#42`? Коротку мітку модель копіює без помилок, вона дешевша в токенах, а мапінг `label → chunk` тримає код, як surrogate key. Розділ і дата дають моделі контекст про походження і свіжість шматка; дата знадобиться в розділі 6. Це та сама ідея contextual header, що й на занятті 6, але для генерації, а не для пошуку.

4.2 Відповідь за схемою

Як на занятті 4, просимо не прозу, а JSON за схемою (structured output з `strict: true`):

```
class Answer(BaseModel):
    found: bool            # false, якщо в джерелах відповіді немає
    answer: str
    citations: list[str]   # мітки джерел: ["S1", "S3"]
```

Поле `found` — явний сигнал «відповіді немає», замість парсингу фраз на кшталт «I'm sorry, I couldn't find». Поле `citations` — список, який можна провалідувати проти множини виданих джерел.

4.3 Валідація цитат

Модель може послатися на `S7`, коли джерел було п'ять, або відповісти без жодної цитати. Тому після генерації код перетинає `citations` з ключами `sources`. Якщо модель повертає `found: true`, а після фільтра не лишилось жодної цитати, відповідь не grounded, і користувач отримує NOT_FOUND. Це звичайна валідація відповіді зовнішнього сервісу, тільки сервіс тут — LLM.

На прогоні ця перевірка зламала нас самих. У першій версії промпту модель на українських питаннях інколи повертала `"citations": ["[S3]"]`. Перевірка не знаходила мітку `[S3]` і відкидала відповідь, тож правильне «25 кг» ставало NOT_FOUND. Виправлення — нормалізувати мітку перед перевіркою і явно описати формат у промпті. Загальний урок: валідатор, який мовчки відкидає відповіді, теж треба моніторити, інакше він ховає регресії (розділ 5.4).

Деякі провайдери цитують на рівні API: Anthropic Citations повертає точні фрагменти документа для кожного речення відповіді, і валідацію міток тоді робить сам API.

4.4 Порядок джерел

Модель використовує довгий контекст нерівномірно. У дослідженні Liu et al. (Lost in the Middle) моделі давали 20 документів, серед яких один з відповіддю. Коли він стояв першим, точність була 75.8%, у середині списку — 53.8%, тобто нижче, ніж без документів узагалі (56.1%). Тому джерела йдуть у порядку релевантності з retrieval, найкраще першим, і контекст не роздувають без потреби.

ГОЛОВНЕ

Кожне джерело має **мітку, документ, розділ і дату**. Модель повертає JSON з `found` і `citations`, а **код валідує** цитати проти виданих джерел: відповідь без справжньої цитати — NOT_FOUND.

Глибше: Anthropic: Citations API · Liu et al.: Lost in the Middle · OpenAI: Structured outputs

---

РОЗДІЛ 5

5 «Не знаю»

5.1 Чому система вигадує

Retrieval завжди щось повертає: top-5 існує навіть для питання, відповіді на яке в корпусі немає. Модель отримує п'ять тематично схожих чанків і, оптимізована бути корисною, складає з них правдоподібну відповідь. Для внутрішнього асистента наслідок прямий: співробітник діє за правилом, якого не існує.

5.2 Чому поріг за скором не працює

Очевидна ідея — поріг: якщо max cosine серед знайдених чанків нижчий за X, повертаємо NOT_FOUND без виклику моделі. На прогоні для питань з відповіддю max cosine був від 0.32 («Who is on extension 240?») до 0.79. Для питань без відповіді — від 0.37 до 0.57. Діапазони перекриваються, тож будь-який поріг дає або false negatives, або false positives.

Питання без відповіді	max cosine	Що повернув retrieval
What is the cut-off time for UPS parcels?	0.57	таблицю cut-off з DPD, DHL, Raben
How many weeks of paid parental leave do employees get?	0.53	політику віддаленої роботи
What were the total operating costs in Q3? (група all_employees)	0.40	правила витрат; фінзвіт відфільтровано ACL
What is the Wi-Fi password in the Kyiv office?	0.37	правила безпеки й VPN

Найнебезпечніший тип — UPS: тема та сама, знайдено правильну таблицю, у ній є час для кожного перевізника, крім UPS. Семантично близько, а відповіді немає. Це видно лише при читанні тексту, тобто рішення має приймати модель, а не поріг.

5.3 Три рівні

Рівень	Що робить	Хто вирішує
Порожній retrieval	жодного дозволеного чанка → NOT_FOUND без виклику моделі	код
Модель	промпт явно дозволяє і просить повертати `found: false`	модель
Валідація цитат	`found: true` без валідної цитати → NOT_FOUND	код

Текст NOT_FOUND задає код, а не модель: «I could not find this in the documents available to you». Так відповідь стабільна і її легко обробити на фронтенді. На прогоні всі чотири питання без відповіді отримали NOT_FOUND (abstain 1.00), зокрема UPS і фінзвіт для користувача без доступу.

5.4 Друга сторона: false abstain

Систему, що на все повертає NOT_FOUND, легко зробити безпечною і марною. Тому, як precision і recall, abstain міряють разом з false abstain — часткою питань з відповіддю, на які система відмовилась відповідати. Перша чернетка промпту v2 мала суворе правило «схожа тема — ще не відповідь». Abstain лишився 1.00, а false abstain виріс з 0.05 до 0.26, correct упав з 0.89 до 0.74. Причини дві: дужки в цитатах (розділ 4.3) і питання про Raben, де правильна відповідь — «послугу скасовано». Після двох правок false abstain знову 0.05. Дивлячись лише на abstain, ми б зарахували регресію як покращення.

ГОЛОВНЕ

Retrieval завжди щось повертає, а **поріг за скором не відділяє** питання без відповіді. NOT_FOUND будують у три рівні: порожній retrieval, `found: false` від моделі, валідація цитат. Міряють пару метрик: **abstain і false abstain**.

РОЗДІЛ 6

6 Версії й конфлікти

6.1 Старий регламент і нова пам'ятка

У регламенті складу 2016 року, розділ 7.4, таблиця: DPD забирає посилки до 16:30, Raben — до 14:00. У тому самому PDF є додаток D, пам'ятка QMS-2017-04: з 18.04.2017 DPD до 17:15, Raben «discontinued», і пам'ятка прямо пише, що замінює таблицю 7.4. Retrieval повертає обидва чанки, тож у контексті дві суперечливі відповіді.

Питання	Промпт v1	Промпт v2
Latest time to hand over parcels to DPD on a weekday?	17:15 ✓	17:15, з приміткою, що пам'ятка QMS-2017-04 замінила попередній час ✓
Weekday cut-off time for Raben groupage?	14:00 ✗ (застаріле)	«The Raben groupage service has been discontinued» ✓

v1 вгадав з DPD, бо пам'ятка стояла в контексті вище за таблицю, а з Raben узяв застаріле значення. v2 додає правило: якщо джерела дають різні значення, бери новіше (пізніша дата або явна заміна) і

назви замінене значення; «скасовано» — теж відповідь. Зміни зафіксовані в `prompts/CHANGELOG.md`, як на занятті 4.

6.2 Метадані теж бувають неправильними

У заголовку чанка з пам'яткою стоїть `effective 2016-03-01`: на занятті 6 весь PDF отримав одну дату, і пам'ятка її успадкувала. Правило «новіша дата перемагає» спрацювало лише тому, що дата є в самому тексті. Це класична проблема денормалізованих даних: значення скопіювали з батьківського запису, і воно розійшлося з реальністю.

Де це виправляти

Промпт тут лише страховка. Надійно це виправляють в ingestion: виділити пам'ятку в окремий документ з власною датою і зв'язком `replaces: 7.4`, а в retrieval відфільтровувати замінені версії, так само як ACL-фільтр на занятті 7.

ГОЛОВНЕ

Коли джерела суперечать, промпт задає правило: **новіше або явно замінююче перемагає**, і відповідь називає замінене значення. Але надійно це вирішують **дані**: коректні дати й зв'язки версій у метаданих і фільтр застарілого в retrieval.

Глибше: Wu, Wu, Zou: ClashEval

РОЗДІЛ 7

7 Контекст як ресурс

7.1 Бюджет токенів

Кожен чанк у контексті оплачується на кожному запиті. Розмір чанків нерівномірний: типовий займає 50–150 токенів, а таблиця допусків приймання з регламенту (розділ 4.3.2) — 832. Тому контекст збирають не як «перші k», а як «перші k, що вміщаються в бюджет»: greedy-пакування за релевантністю. Чанк, що не влазить, пропускаємо, а менший після нього ще може ввійти. У практиці бюджет 1500 токенів, рахуємо тим самим токенізатором, що й модель (`tiktoken`, `o200k_base`).

7.2 Скільки чанків брати

Ми прогнали всі 23 питання з k = 3, 5 і 10. Щоб бюджет не обрізав контекст при більшому k, у цьому порівнянні він масштабується з k: 400 токенів на чанк (1200, 2000 і 4000).

k	retrieved	correct	cited	abstain	токенів на запит	$ / 1000 запитів
3	0.95	0.95	0.95	1.00	666	0.12
5	0.95	0.95	0.95	1.00	854	0.15
10	1.00	0.95	0.95	1.00	1256	0.21

Якість відповіді не змінилась, вартість від k=3 до k=10 зросла на 74%. При k=10 retrieved досягає 1.00: чанк q09 потрапляє в контекст, але відповідь лишається неправильною, бо проблема в даних (розділ 10.2). Latency майже не залежить від k (1.2–1.3 с у всіх трьох): prefill вхідних токенів

швидкий, а час іде на генерацію відповіді. На цьому корпусі достатньо k=3. Але це висновок для 69 чанків і 23 питань: на вашому корпусі правильний чанк може стояти 6-м, і k=3 його втратить. Тому k — це параметр, який підбирають прогоном на своєму наборі, як будь-який конфіг продуктивності, а не беруть з туторіалу.

7.3 Чому не покласти все в long context

Вікна сучасних моделей — сотні тисяч токенів, тож виникає спокуса пропустити retrieval. Крім ціни, заважає якість. Databricks перевірила RAG з різною довжиною контексту: recall росте до 0.95, а якість відповіді в більшості моделей починає падати після 4–64 тисяч токенів, залежно від моделі. GPT-4o і `gpt-4o-mini` у тому тесті майже не деградували, тож межу треба міряти для своєї моделі.

Компромісний патерн — fallback: спершу RAG, і лише коли він повертає «не знайшов», повторити запит з long context. У дослідженні Google DeepMind (Self-Route) GPT-4o з такою маршрутизацією відповідав не гірше, ніж з long context завжди, на 61% токенів, а Gemini-1.5-Pro — трохи гірше, на 38%.

ГОЛОВНЕ

Контекст — це **бюджет**: токени, гроші, latency і увага моделі. k обирають **прогоном** на своєму наборі питань: у практиці k=3 дав ту саму якість, що й k=10, на 43% дешевше.

Глибше: Databricks: Long context RAG performance · Li et al.: Self-Route

---

РОЗДІЛ 8

8 Більше, ніж вікно

8.1 Запит, на який top-k не відповідає

«List every rule about temperature in the warehouse manual». Правила розкидані по регламенту: приймання (щуп між коробками, 30 секунд; якщо три заміри різняться більше ніж на 3 °C, міряти ще дві коробки), допуски (відмовити, якщо охолоджений товар тепліший за +8 °C), зберігання (логери кожні 10 хвилин), охорона праці (костюм у морозильній камері не більше 45 хвилин на годину). Ми виписали 8 таких фактів з різних розділів як еталон. Відповідь на основі top-5 містить 2 з 8: retrieval оптимізований знаходити найрелевантніші шматки, а не всі релевантні.

8.2 Map-reduce

Патерн той самий, що в MapReduce над даними. Документ ділимо на вікна, що вміщаються в контекст. Map: модель читає одне вікно і повертає факти, релевантні запиту, з id чанків, або NONE. Reduce: один виклик зливає нотатки, прибирає дублікати й формує відповідь. Map-виклики незалежні, тож у продакшні їх паралелять; у практиці для простоти вони послідовні.

Метод	фактів	викликів	вхідних токенів	$	секунд
top-5	2 / 8	1	905	0.00019	1.8
map-reduce (4 вікна по 1500 токенів)	6 / 8	5	6814	0.00163	17.6

Утричі більше фактів, але у 8.6 раза дорожче і майже в 10 разів довше (послідовно; з паралельним map latency була б ближче до двох викликів). Повноти він теж не гарантує: пропустив логери кожні 10 хвилин і ліміт для морозильної камери, хоча top-5 логери знайшов. Обидва факти стоять у розділах, де температура не головна тема, і на кроці map модель їх не виписала. Регламент займає 4552 токени і насправді влазить у вікно `gpt-4o-mini`; вікна по 1500 ми задали штучно, щоб показати механіку, потрібну для документів на сотні сторінок.

8.3 Інші підходи

Підхід	Ідея	Коли
Більше k + reranker	більше кандидатів, у контекст лише найкращі	відповідь у кількох місцях
Map-reduce	пройти весь документ вікнами, потім злити	«перелічи все», «підсумуй документ»
Ієрархічні підсумки	на індексації зробити підсумки розділів і документів і шукати по них	питання про весь корпус
Long context як fallback	спершу RAG, довгий контекст — коли RAG каже «не знайшов»	рідкісні складні запити

Microsoft GraphRAG — варіант ієрархічних підсумків: на індексації LLM будує граф сутностей, групує його в спільноти й пише підсумок для кожної. На запитах про весь корпус такі відповіді перемагали звичайний RAG за повнотою у 72–83% порівнянь. Ціну платять наперед: індексація викликає модель на кожен шматок корпусу, як матеріалізоване представлення, яке дорого будувати, але дешево читати.

ГОЛОВНЕ

Top-k відповідає на запити, відповідь на які лежить у кількох місцях. Для «перелічи все» потрібен **map-reduce** або **підсумки, зроблені на індексації**. Обидва дорожчі на порядок, тому їх вмикають для окремого класу запитів, а не для всіх.

Глибше: Edge et al.: GraphRAG

РОЗДІЛ 9

9 Діалог і пам'ять

9.1 «What is the yearly limit for that?»

У чаті користувачі не повторюють контекст. Після відповіді «Yes. Home office equipment such as a keyboard, mouse or headset can be claimed» співробітник питає: «What is the yearly limit for that?» Retrieval отримує лише останнє повідомлення, де немає ні «keyboard», ні «equipment»: займенник «that» посилається на попередній хід розмови, якого пошук не бачить.

Уточнення	Без переписування	З переписуванням
What is the yearly limit for that?	«The yearly limit for training is 600 EUR per year» ✗	запит став «What is the yearly limit for claiming money for home office equipment?» → 40 EUR ✓
And what is their e-mail? (після питання про контакт QA)	NOT_FOUND ✗	запит став «And what is Dr. L. Meier's e-mail?» → qa@northwind.example ✓

Перший випадок гірший за NOT_FOUND: впевнена відповідь з валідною цитатою, але на інше питання. Цікаво, що retrieval тут не винен: без переписування retrieved той самий, 0.95, і чанк з лімітом 40 EUR потрапив у контекст поруч з таблицею лімітів. Але модель отримала той самий голий запит і не знала, на що вказує «that», тож обрала рядок, що пасує до «yearly limit». Валідація цитат такого не ловить, бо цитата справжня.

9.2 Condense question

Перед retrieval окремий виклик моделі переписує follow-up у самостійний запит з урахуванням історії: розв'язує займенники, зберігає мову, нічого не відповідає. Виклик робимо лише за наявності історії, тож перше питання в сесії не платить за нього. На всьому наборі це підняло correct з 0.84 до 0.95: обидва уточнювальні питання стали правильними. Ціна — ще один виклик моделі, тобто додаткова latency і токени на кожен follow-up.

Переписаний запит іде і в retrieval, і в generate. На нашому маленькому корпусі пошук випадково впорався й без нього, а зламалась генерація; на більшому корпусі «that» може зламати і пошук. Історію можна передавати в generate напряму, але це дорожче (розділ 9.3), а retrieval однаково отримує лише рядок запиту. Довгострокова пам'ять агента між сесіями — заняття 13.

9.3 Історія як ресурс

Пам'ять діалогу конкурує за той самий бюджет, що й джерела. Якщо кожен хід додає t токенів і кожен запит несе всю попередню історію, n-й запит містить близько n·t токенів історії, а сумарна вартість сесії росте квадратично від кількості ходів. Тому історією керують так само явно, як контекстом:

- generate отримує переписаний запит, а не всю розмову; історію бачить лише condense (так влаштовано в практиці);
- condense отримує вікно останніх N ходів, а не всю сесію;
- старішу частину розмови стискають у підсумок, який оновлюють раз на кілька ходів;
- на історію виділяють окремий бюджет токенів поруч з бюджетом на джерела, і при переповненні першою обрізають історію, а не джерела.

У практиці історія займає один хід, тож ці прийоми ми не міряли. Для свого продукту перевіряйте їх тим самим способом: набір діалогів з уточненнями і метрики correct, токени й latency до і після.

ГОЛОВНЕ

Follow-up на кшталт «а для цього?» треба **переписати в самостійний запит** до пошуку і генерації, інакше система впевнено відповість на інше питання з валідною цитатою. Історія діалогу — частина **бюджету контексту**: вікно останніх ходів, підсумок старіших, окремий ліміт токенів.

РОЗДІЛ 10

10 Де зламалось

10.1 Трасування по етапах

Неправильна відповідь не каже, де помилка, як і 500 без логів. Тому для кожного запиту практика пише в `output/answers-*.jsonl` структурований запис: переписаний запит, `retrieved`, `in_context`, `cited`, `answer`. Це той самий підхід, що трасування зі span на кожен етап: за цими полями помилку однозначно відносять до етапу. Агреговано це видно в метриках: у фінальній версії retrieved 0.95 і correct 0.95, тобто помилки лише в пошуку; без переписування уточнень retrieved лишається 0.95, а correct падає до 0.84, тобто нові помилки — в генерації.

Що в записі	Етап	Що робити
правильного чанка немає в `retrieved`	retrieval	hybrid, k, reranker, переписування запиту (заняття 7, розділ 9)
є в `retrieved`, немає в `in_context`	бюджет	більший бюджет, менші чанки
є в `in_context`, відповідь хибна чи NOT_FOUND	generation або дані	промпт, chunking, метадані
відповідь правильна, метрика каже «ні»	оцінка	еталон чи scoring-функція (заняття 9)

10.2 Розбір: угода BV-2013-02

Фінальна версія помиляється лише на одному питанні: «What is agreement BV-2013-02 about?» На занятті 7 запит «agreement BV-2013-02» BM25 знаходив першим. Тут BM25 теж ставить потрібний чанк першим, але у векторному пошуку він лише 30-й, і RRF дає йому 7-ме місце. При k=5 він не потрапляє в контекст: помилка retrieval.

При k=10 чанк уже в контексті (retrieved 1.00), а модель однаково повертає NOT_FOUND. Причина в даних: про угоду сказано лише у виносці («2 Agreed with the works council, see agreement BV-2013-02»), а маркер виноски злипся зі словом «decisions2» в основному тексті. Модель не пов'язує виноску з реченням, до якого вона належить. Виправлення не в промпті, а в парсері PDF (заняття 6): підставляти текст виноски поруч із місцем, де на неї посилаються.

Окремий клас помилок — в оцінці. На питання про reason code 14 модель відповіла «writing off a frozen carton … with the drill probe». Це правильно, але еталон вимагав слова «sacrificial», і перевірка спершу зарахувала відповідь як хибну. Детерміновані перевірки за ключовими словами дешеві, але буквальні; на занятті 9 до них додамо LLM-as-a-judge.

ГОЛОВНЕ

Неправильна відповідь — це симптом. Логуйте **запит, retrieved, in_context, cited і answer** і відносіть кожну помилку до етапу: retrieval, бюджет, дані, генерація чи **сама оцінка**. Найчастіше виправляти треба не промпт.

РОЗДІЛ 11

11 Перевірте себе

> Спробуйте відповісти без підглядання. Якщо питання дається важко, поверніться до розділу в дужках.

1. Чому не можна просто покласти всі документи компанії в промпт? Назвіть три причини. *(1.1, 7.3)*
2. Чому fine-tuning погано додає нові факти? Назвіть три вимоги до системи з політиками компанії, які робить простішими RAG. *(2.1–2.2)*
3. Які етапи пайплайна викликають модель, а які ні? Як це впливає на тестування? *(3.1)*
4. retrieved 0.95, correct 0.84. Де шукати причину помилок і чи допоможе reranker? Коли варто додавати multi-query чи HyDE? *(3.3)*
5. Навіщо мітки `S1` замість `chunk_id` і навіщо в заголовку джерела дата? *(4.1)*
6. Модель повернула `found: true` і `citations: ["S7"]`, а джерел було п'ять. Що має повернути система? *(4.3)*
7. Де в контексті має стояти найрелевантніший чанк і чому? *(4.4)*
8. Чому поріг «max cosine < 0.45 → NOT_FOUND» не працює? Наведіть питання, яке його обходить. *(5.2)*
9. Ви посилили промпт, і abstain виріс з 0.75 до 1.00. Яку метрику треба перевірити перед мерджем? *(5.4)*
10. Регламент каже 16:30, пам'ятка — 17:15. Що має бути в промпті, а що в даних? *(6.1–6.2)*
11. k=3 і k=10 дали однакову якість. Яке рішення приймете і що перевірите на своєму корпусі? *(7.2)*
12. Чому map-reduce знайшов більше фактів, ніж top-5, і чому його не вмикають для всіх запитів? Як зменшити його latency? *(8.2)*
13. Чому відповідь на follow-up без переписування може мати валідну цитату і бути неправильною? *(9.1)*
14. Чому вартість сесії чату росте швидше, ніж кількість ходів, і які три прийоми це обмежують? *(9.3)*
15. Чанк є в `in_context`, а модель повертає NOT_FOUND. Де шукати причину? *(10.1–10.2)*

12 Перед заняттям

> На занятті працюємо з кодом з репозиторію курсу. Підготуйте середовище й ключ заздалегідь.

Репозиторій, бібліотеки, моделі, ключ

```
git pull                                    # або git clone https://github.com/andbilous/ai-engineeering-2
cd ai-engineeering-2/lesson-08-rag
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# модель embeddings з заняття 7 (~220 МБ, один раз, у папку .models)
python -c "from fastembed import TextEmbedding; \
TextEmbedding('sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2',
cache_dir='.models')"

cp .env.example .env                        # і вставте свій OPENROUTER_API_KEY

# словник токенізатора для tiktoken (завантажується один раз, потрібна мережа)
python -c "import tiktoken; tiktoken.get_encoding('o200k_base')"
```

Відповіді генерує `openai/gpt-4o-mini` через OpenRouter, як на заняттях 3–4. Повний прогін 23 питань коштує менше цента. Без ключа можна працювати з локальною моделлю через Ollama (закоментований рядок у `rag.py`), але маленькі моделі гірше дотримуються формату цитат. Цифри в цьому тексті отримано на `gpt-4o-mini`. Пошук (embeddings, Qdrant у пам'яті, BM25) працює локально, як на занятті 7. Потрібен Python 3.10–3.12.

Файл	Що це
`retrieval.py`	готовий hybrid search з заняття 7 (розв'язок), змінювати не треба
`data/chunks.jsonl`	69 чанків Northwind з заняття 7
`data/questions.json`	23 питання: факти, яких чекаємо у відповіді, правильні чанки, тип (перефраз, код, українською, конфлікт версій, без відповіді, уточнення)
`prompts/`	`v1/system.txt`, готові допоміжні промпти у `v2/`, `CHANGELOG.md`
`starter/rag.py`	пайплайн з TODO 1–9 (TODO 2 вже написаний)
`starter/check.py`	10 перевірок: 9 з фейковою моделлю (без ключа) і одна з реальною

Домашня частина: TODO 1–5

Вдома збираєте базовий RAG: контекст, перевірку цитат, «не знайшов» і метрики. Виклик моделі (TODO 2) уже написаний: це structured output з заняття 4, прочитайте його перед TODO 3. На занятті додаємо промпт v2, вибір k, map-reduce і уточнювальні питання. Пишіть код з AI-асистентом, але кожен рядок маєте вміти пояснити: на занятті хтось із вас покаже свій розв'язок.

Перед кожним TODO в коді є короткий вступ: мета, навіщо і результат, за яким видно, що задачу зроблено. Та сама логіка зібрана в таблицях нижче. Якщо слухаєте заняття паралельно з кодом, починайте з колонки «Навіщо».

TODO	Що зробити	Навіщо	Результат
1	`build_context()`: джерела S1…Sn із заголовком (документ, розділ, дата) у межах бюджету токенів	модель цитує лише те, що має мітку; кожен токен коштує (4.1, 7.1)	перевірка 1: великий чанк пропущено, менший після нього ввійшов
2	готово: `generate()` — промпт + джерела + питання, виклик з `ANSWER_FORMAT`, розбір у `Answer`. Прочитайте	JSON дає коду `found` і цитати замість прози (4.2)	перевірка 2 проходить одразу
3	`validate_citations()`: нормалізувати «[S1]», відкинути неіснуючі мітки	відповідь без справжньої цитати ні на чому не тримається (4.3)	перевірка 3
4	`rag()`: пошук → контекст → генерація → перевірка → «не знайшов»	вигадана відповідь гірша за відсутню (5.3)	перевірка 4; питання без відповіді дають NOT_FOUND
5	`evaluate()`: retrieved, in_context, correct, cited, abstain, false abstain	без цифр не видно, чи допомагає наступна зміна (5.4)	таблиця з рядком `v1 0.84`

На занятті: TODO 6–9

TODO	Що зробити	Навіщо	Результат
6	промпт `v2/system.txt` + запис у CHANGELOG	застарілі значення і зайві відмови (5.4, 6.1)	питання про Raben стає правильним
7	`sweep()`: k = 3, 5, 10	обрати розмір контексту за цифрами (7.2)	таблиця якість / токени / $
8	`map_reduce()` по всьому регламенту	питання «перелічи все» (8.2)	`--summary`: факти top-5 проти map-reduce
9	`condense()`: переписати уточнення	пошук бачить лише рядок запиту (9)	обидва уточнювальні питання правильні

Запуск:

```
python starter/rag.py                                  # 23 питання + таблиця метрик
python starter/rag.py "How much can I spend on a hotel?"   # одне питання з джерелами
python starter/check.py
```

Що побачите

```
      retr   ctx  correct  cited  abstain  f.abst  tokens in  $/1000  ...   failed
v1    0.95  0.95     0.84   0.84     1.00    0.11        653   0.117  ...   q09 q16 q23
FOR CHAT: openai/gpt-4o-mini v1 k=5 | correct 0.84 | cited 0.84 | abstain 1.00 | $0.12/1000 q
| ...
6/10 checks passed
```

Без змін у коді скрипт зупиниться з `Not implemented yet`, а `check.py` покаже `1/10`: проходить лише перевірка готового TODO 2. Після TODO 1, 3–5 буде `6/10`: п'ять офлайн-перевірок і live-перевірка з

реальною моделлю (без ключа її пропущено, і результат буде `5/9`). У таблиці провалені q09, q16 і q23. Ці три питання ми й розберемо на занятті: q16 виправить промпт v2, q23 — переписування уточнень, а q09 залишиться прикладом помилки в даних (розділ 10.2). Час на питання у вас буде інший.